

ELI — Diagrams

ELI v2.4.71

September 2026

Contents

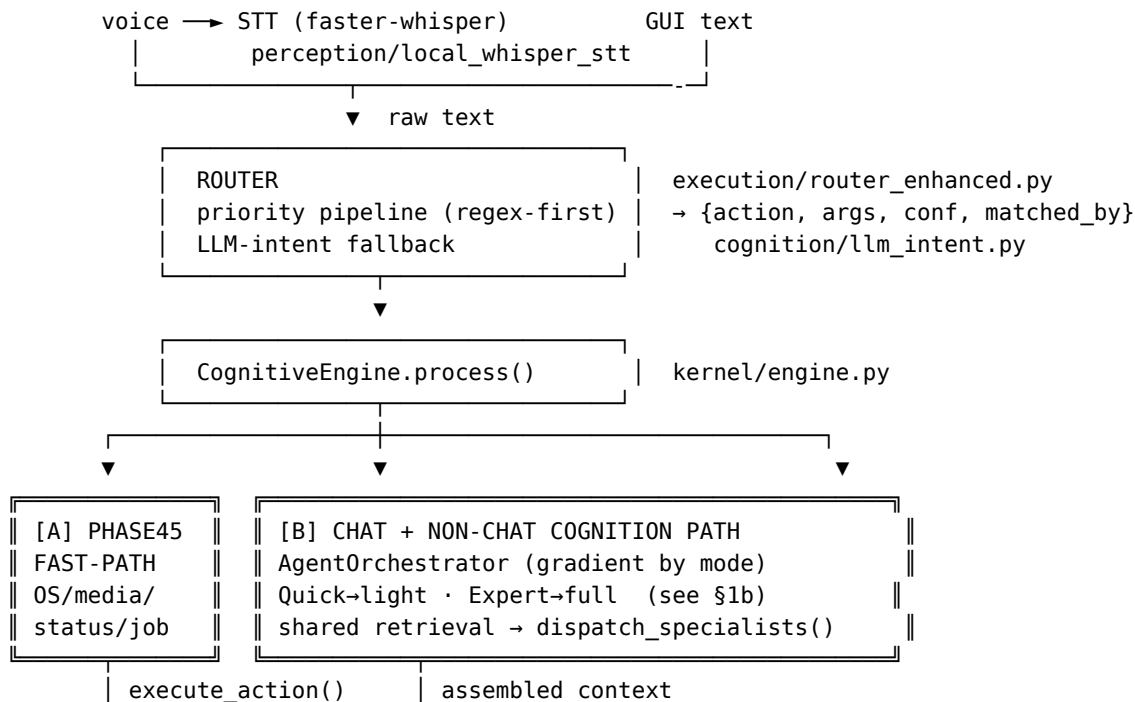
Blueprint — ELI ASCII Diagrams	1
1. Full pipeline (request lifecycle)	1
2. Memory subsystem eli/memory + cognition/working_memory.py	2
3. Gating stack (in path order — every guard the request passes)	3
4. One-screen overview (how it all connects)	3

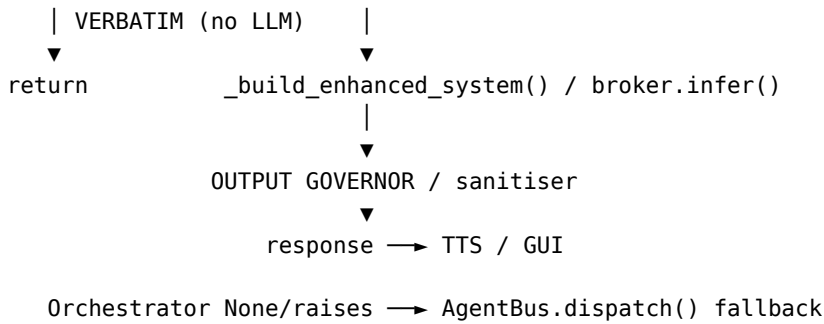
Blueprint — ELI ASCII Diagrams

Updated for v2.4.71. CHAT uses a gradient orchestrator for all modes; retrieval is unified in eli/memory/retrieval.py; Stage 12 = learning/state commit.

Visual companion to architecture.md. Three views: the full request pipeline, the memory subsystem, and the gating stack. All grounded in the real modules (file paths are clickable).

1. Full pipeline (request lifecycle)

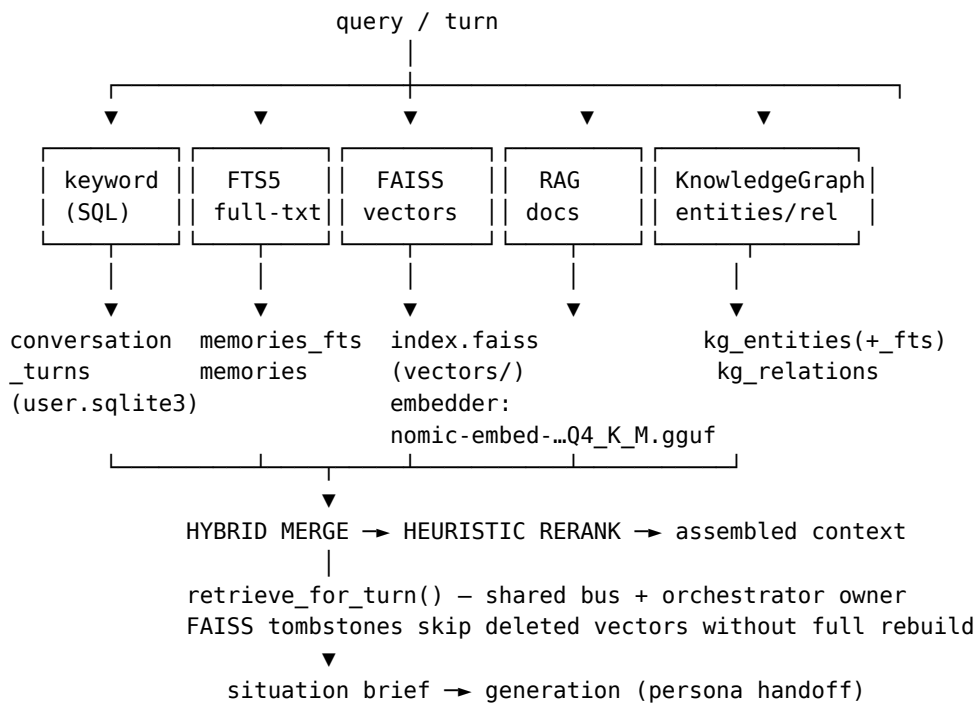




1b. Orchestrator — canonical S01-S12 (pipeline_trace.py)

S01 PERCEIVE → S02 INPUT_GUARDS → S03 ROUTER → S04 GROUNDING_GATE
 → S05 PLANNER (mode-aware budgets)
 → S06 AGENT_BUS: retrieve_for_turn() + dispatch_specialists()
 → S07 CONTEXT_ASSEMBLY (merge + heuristic rerank)
 → S08 INFERENCE_BROKER → S09 REASONING_SYNTHESIS (mode algo)
 → S10 OUTPUT_GOVERNOR → S11 RESPONSE_DELIVERY
 → S12 LEARNING_STATE_UPDATE (learning_coordinator.finalize_turn)

2. Memory subsystem eli/memory + cognition/working_memory.py



STORES (artifacts/)	
db/user.sqlite3	memories, memories_archive, memory_meta, semantic, conversation_turns, conversations, session_summaries, kg_entities(+_fts), kg_relations, recall_log, runtime_events, learning_replay, observations, habits/habit_events/habit_rules, user_patterns, user_model, eli_stances, belief_revisions, news_articles(+_fts), news_reflections

db/agent.sqlite3	agent_dispatches, agent_metrics, improvements, failures, corrections, capability_proposals, error_tracking (self-improvement lives here only)
db/system_index.sqlite3	· db/coding_memory.sqlite3
vectors/index.faiss	semantic index

A question that names a period ("last week") is filtered by date FIRST (Memory.memories_between + dated conversation turns), then ranked by topic.

WRITE PATH: turn → PERSISTENCE GATE (drop junk/report-dumps) → STORAGE POLICY (origin · dedupe · importance) → store; daily upkeep decays and archives

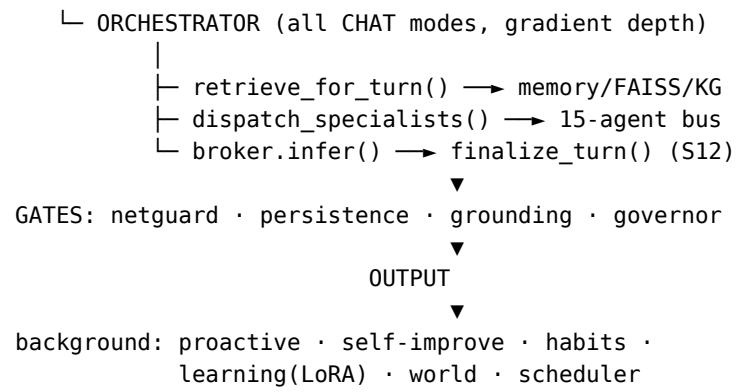
3. Gating stack (in path order — every guard the request passes)

1	NETGUARD (process-wide, always-on)	core/netguard.py	
	any outbound socket → offline? → OfflineError (FAIL-CLOSED)		
	allow_network(): scoped opt-in window (model download, web tier)		
2	PHASE45 FAST-PATH GATE	kernel/engine.py	
	deterministic action (volume/media/date/job/status)?		
	→ execute_action() VERBATIM, no LLM (cannot confabulate)		
3	PERSISTENCE GATE	runtime/persistence_gate.py	
	about to write to memory? junk / internal report-dump?		
	→ DROP (so it can't be stored and replayed as fact later)		
4	GROUNDING GATE	runtime/deterministic_gate	
	answer requires evidence? unbacked or raw packet?		
	→ BLOCK raw evidence/telemetry leak; require grounded synthesis		
5	GROUNDING ESCALATION	runtime/grounding_escalation	
	checkable fact AND grounding < threshold ?		
	external fact → WEB tier		
	self/project → LOCAL tier		
	exhausted / offline → HEDGE ("I won't guess")		
	(trigger = GROUNDING, not the response score – which lies when wrong)		
6	LEARNING / STATE COMMIT (S12)	learning_coordinator.py	
	finalize_turn(): store assistant turn · publish meta · learn from result		
7	OUTPUT GOVERNOR / SANITISER	cognition/output_governor.py	
	final text → strip leaks, persona hygiene, no raw JSON → user		

Legend: FAIL-CLOSED = denies by default; VERBATIM = returned without an LLM pass; HEDGE = honest "can't verify" instead of a guess.

4. One-screen overview (how it all connects)

INPUT → ROUTER → ENGINE — FAST-PATH —→ (verbatim) → OUTPUT



““